```
"""
CAMPUS CONNECT — FYCS (First Year Computer Science)
Sheth L.U.J. College of Arts & Sir M.V. College of Science & Commerce
Semester I Lecture Timetable  A.Y. 2025-26  |  Date: 22-11-2025
SQLite-backed management system
"""

import tkinter as tk
from tkinter import ttk, messagebox
from datetime import datetime
import sqlite3
import hashlib

#  DATABASE

class DatabaseManager:
    def __init__(self, db_path="fycs_campus.db"):
        self.db_path = db_path
        self.conn = sqlite3.connect(self.db_path)
        self.conn.row_factory = sqlite3.Row
        self._create_tables()
        self._seed()

    def _create_tables(self):
        self.conn.executescript("""
            CREATE TABLE IF NOT EXISTS students (
                id          TEXT PRIMARY KEY,
                name        TEXT NOT NULL,
                phone       TEXT DEFAULT '',
                year        TEXT DEFAULT 'FY',
                department  TEXT DEFAULT 'CS',
                roll_no     TEXT DEFAULT '',
                password    TEXT NOT NULL,
                created_at  TEXT DEFAULT CURRENT_TIMESTAMP
            );
            CREATE TABLE IF NOT EXISTS announcements (
                id         INTEGER PRIMARY KEY AUTOINCREMENT,
                message    TEXT NOT NULL,
                author     TEXT DEFAULT 'Admin',
                created_at TEXT DEFAULT CURRENT_TIMESTAMP
            );
            CREATE TABLE IF NOT EXISTS assignments (
                id          INTEGER PRIMARY KEY AUTOINCREMENT,
                student_id   TEXT NOT NULL,
```

```
        subject      TEXT NOT NULL,
        title        TEXT NOT NULL,
        filename     TEXT NOT NULL,
        submitted_at TEXT DEFAULT CURRENT_TIMESTAMP,
        FOREIGN KEY (student_id) REFERENCES students(id)
    );
    CREATE TABLE IF NOT EXISTS lab_bookings (
        id           INTEGER PRIMARY KEY AUTOINCREMENT,
        student_id   TEXT NOT NULL,
        student_name TEXT NOT NULL,
        software     TEXT NOT NULL,
        purpose      TEXT NOT NULL,
        duration     INTEGER NOT NULL,
        status       TEXT DEFAULT 'waiting',
        booked_at    TEXT DEFAULT CURRENT_TIMESTAMP,
        FOREIGN KEY (student_id) REFERENCES students(id)
    );
    CREATE TABLE IF NOT EXISTS grades (
        id         INTEGER PRIMARY KEY AUTOINCREMENT,
        student_id TEXT NOT NULL,
        subject    TEXT NOT NULL,
        marks      REAL NOT NULL,
        max_marks  REAL NOT NULL DEFAULT 100,
        updated_at TEXT DEFAULT CURRENT_TIMESTAMP,
        UNIQUE(student_id, subject),
        FOREIGN KEY (student_id) REFERENCES students(id)
    );
  """)
  self.conn.commit()

def _seed(self):
  pw = self._hash("password")
  students = [
    ("CS001", "Aarav Mehta",    "9876501001", "FY", "CS", "2301"),
    ("CS002", "Priya Sharma",   "9876501002", "FY", "CS", "2302"),
    ("CS003", "Rohan Desai",    "9876501003", "FY", "CS", "2303"),
    ("CS004", "Sneha Patil",    "9876501004", "FY", "CS", "2304"),
    ("CS005", "Arjun Nair",     "9876501005", "FY", "CS", "2305"),
    ("CS006", "Kavya Iyer",     "9876501006", "FY", "CS", "2306"),
    ("CS007", "Rahul Verma",    "9876501007", "FY", "CS", "2307"),
    ("CS008", "Pooja Joshi",    "9876501008", "FY", "CS", "2308"),
  ]
  for sid, name, phone, year, dept, roll in students:
    self.conn.execute(
```

```
            "INSERT OR IGNORE INTO students "
            "(id,name,phone,year,department,roll_no,password) VALUES (?,?,?,?,?,?,?)",
            (sid, name, phone, year, dept, roll, pw)
        )

    announcements = [
        ("Web Technologies Practical Batch-1 shifted to Lab 01 on Monday",       "Admin",
"2025-11-22 08:00"),
        ("DAA Unit Test scheduled for next Thursday — syllabus: Unit 1 & 2",      "Admin",
"2025-11-21 14:00"),
        ("Adv. Python Programming project submission deadline: 30-Nov-2025",       "Admin",
"2025-11-20 10:30"),
        ("Library will remain closed on Saturday 29-Nov-2025",                    "Admin",
"2025-11-19 09:00"),
        ("Guest lecture on Research Methodology — Dr. Mahendra Kanojia — Wed",    "Admin",
"2025-11-18 11:00"),
    ]
    for msg, author, ts in announcements:
        self.conn.execute(
            "INSERT OR IGNORE INTO announcements (message,author,created_at) VALUES
(?,?,?)",
            (msg, author, ts)
        )

    # Only seed assignments if the table is empty (avoids duplicates on restart)
    existing = self.conn.execute("SELECT COUNT(*) FROM assignments").fetchone()[0]
    if existing == 0:
        assignments = [
            ("CS001","Web Technologies",                "HTML5 Portfolio Project",      "portfolio.html",
"2025-11-20 14:30"),
            ("CS001","Object Oriented Programming",     "Inheritance Lab Program",
"inheritance.java",   "2025-11-19 16:00"),
            ("CS001","Advanced Python Programming",     "Data Analysis Script",
"data_analysis.py",   "2025-11-18 11:45"),
            ("CS001","Design & Analysis of Algorithms", "Sorting Comparison Report",
"sorting_report.pdf", "2025-11-17 09:30"),
            ("CS002","Web Technologies",                "CSS Grid Layout",              "layout.css",
"2025-11-20 15:00"),
            ("CS002","Advanced Python Programming",     "NumPy Arrays Demo",
"numpy_demo.py",      "2025-11-19 12:00"),
            ("CS003","Object Oriented Programming",     "Polymorphism Example",
"poly.java",          "2025-11-18 10:00"),
        ]
        for sid, subj, title, fname, ts in assignments:
```

```
            self.conn.execute(
                "INSERT INTO assignments "
                "(student_id,subject,title,filename,submitted_at) VALUES (?,?,?,?,?)",
                (sid, subj, title, fname, ts)
            )

        existing_bookings = self.conn.execute("SELECT COUNT(*) FROM
lab_bookings").fetchone()[0]
        if existing_bookings == 0:
            bookings = [
                ("CS002","Priya Sharma","VS Code + Python","Advanced Python Project",
3,"waiting","2025-11-22 09:15"),
                ("CS003","Rohan Desai", "Eclipse IDE",     "OOP Lab Assignment",
2,"waiting","2025-11-22 10:00"),
                ("CS004","Sneha Patil", "NetBeans",        "Web Technologies Practical",
2,"waiting","2025-11-22 10:45"),
            ]
            for sid, sname, sw, pur, dur, status, ts in bookings:
                self.conn.execute(
                    "INSERT INTO lab_bookings "
                    "(student_id,student_name,software,purpose,duration,status,booked_at) VALUES
(?,?,?,?,?,?,?)",
                    (sid, sname, sw, pur, dur, status, ts)
                )

        grades = [
            ("CS001","Web Technologies",                78, 100),
            ("CS001","Object Oriented Programming",     85, 100),
            ("CS001","Advanced Python Programming",     91, 100),
            ("CS001","Design & Analysis of Algorithms", 74, 100),
            ("CS001","Basic Concepts in Research",      88, 100),
            ("CS001","Descriptive Statistics",          82, 100),
            ("CS001","Content Writing",                 79, 100),
            ("CS001","Hindi Bhasha",                    72, 100),
            ("CS002","Web Technologies",                80, 100),
            ("CS002","Object Oriented Programming",     76, 100),
            ("CS003","Advanced Python Programming",     88, 100),
            ("CS003","Design & Analysis of Algorithms", 65, 100),
        ]
        for sid, subj, marks, mx in grades:
            self.conn.execute(
                "INSERT OR IGNORE INTO grades (student_id,subject,marks,max_marks) VALUES
(?,?,?,?)",
                (sid, subj, marks, mx)
```

```
        )

    self.conn.commit()

    @staticmethod
    def _hash(pw): return hashlib.sha256(pw.encode()).hexdigest()

    #  Students
    def get_student(self, sid):
        return self.conn.execute("SELECT * FROM students WHERE id=?", (sid,)).fetchone()

    def get_all_students(self):
        return self.conn.execute("SELECT * FROM students ORDER BY id").fetchall()

    def search_students(self, q):
        w = f"%{q}%"
        return self.conn.execute(
            "SELECT * FROM students WHERE id LIKE ? OR name LIKE ? OR department LIKE ?
OR roll_no LIKE ?",
            (w, w, w, w)
        ).fetchall()

    def add_student(self, sid, name, phone, year, dept, roll, password="password"):
        try:
            self.conn.execute(
                "INSERT INTO students (id,name,phone,year,department,roll_no,password) VALUES
(?,?,?,?,?,?,?)",
                (sid, name, phone, year, dept, roll, self._hash(password))
            )
            self.conn.commit()
            return True, "Registered successfully!"
        except sqlite3.IntegrityError:
            return False, "Student ID already exists!"

    def update_student(self, sid, name, phone, year, dept, roll):
        self.conn.execute(
            "UPDATE students SET name=?,phone=?,year=?,department=?,roll_no=? WHERE
id=?",
            (name, phone, year, dept, roll, sid)
        )
        self.conn.commit()

    def delete_student(self, sid):
        self.conn.execute("DELETE FROM students WHERE id=?", (sid,))
```

```
        self.conn.commit()

    def count_students(self):
        return self.conn.execute("SELECT COUNT(*) FROM students").fetchone()[0]

    #  Announcements
    def get_announcements(self):
        return self.conn.execute(
            "SELECT * FROM announcements ORDER BY created_at DESC"
        ).fetchall()

    def add_announcement(self, message, author):
        self.conn.execute(
            "INSERT INTO announcements (message,author) VALUES (?,?)", (message, author)
        )
        self.conn.commit()

    def delete_announcement(self, aid):
        self.conn.execute("DELETE FROM announcements WHERE id=?", (aid,))
        self.conn.commit()

    def count_announcements(self):
        return self.conn.execute("SELECT COUNT(*) FROM announcements").fetchone()[0]

    #  Assignments
    def get_assignments(self, sid):
        return self.conn.execute(
            "SELECT * FROM assignments WHERE student_id=? ORDER BY submitted_at
DESC", (sid,)
        ).fetchall()

    def get_all_assignments(self):
        return self.conn.execute(
            """SELECT a.*, s.name as student_name FROM assignments a
              JOIN students s ON a.student_id = s.id
              ORDER BY a.submitted_at DESC"""
        ).fetchall()

    def add_assignment(self, sid, subject, title, filename):
        self.conn.execute(
            "INSERT INTO assignments (student_id,subject,title,filename) VALUES (?,?,?,?)",
            (sid, subject, title, filename)
        )
        self.conn.commit()
```

```
    def delete_assignment_by_id(self, aid):
        self.conn.execute("DELETE FROM assignments WHERE id=?", (aid,))
        self.conn.commit()

    def delete_last_assignment(self, sid):
        row = self.conn.execute(
            "SELECT id FROM assignments WHERE student_id=? ORDER BY id DESC LIMIT 1",
(sid,)
        ).fetchone()
        if row:
            self.conn.execute("DELETE FROM assignments WHERE id=?", (row["id"],))
            self.conn.commit()
            return True
        return False

    def count_assignments(self):
        return self.conn.execute("SELECT COUNT(*) FROM assignments").fetchone()[0]

    # Lab Bookings
    def get_waiting_bookings(self):
        return self.conn.execute(
            "SELECT * FROM lab_bookings WHERE status='waiting' ORDER BY booked_at"
        ).fetchall()

    def get_all_bookings(self):
        return self.conn.execute(
            "SELECT * FROM lab_bookings ORDER BY booked_at DESC"
        ).fetchall()

    def add_booking(self, sid, sname, software, purpose, duration):
        self.conn.execute(
            "INSERT INTO lab_bookings (student_id,student_name,software,purpose,duration)
VALUES (?,?,?,?,?)",
            (sid, sname, software, purpose, duration)
        )
        self.conn.commit()

    def process_next_booking(self):
        row = self.conn.execute(
            "SELECT * FROM lab_bookings WHERE status='waiting' ORDER BY booked_at LIMIT
1"
        ).fetchone()
        if row:
```

```
            self.conn.execute(
                "UPDATE lab_bookings SET status='completed' WHERE id=?", (row["id"],)
            )
            self.conn.commit()
            return dict(row)
        return None

    def delete_booking(self, bid):
        self.conn.execute("DELETE FROM lab_bookings WHERE id=?", (bid,))
        self.conn.commit()

    def count_waiting_bookings(self):
        return self.conn.execute("SELECT COUNT(*) FROM lab_bookings WHERE
status='waiting'").fetchone()[0]

    #  Grades
 def get_grades(self, sid):
        return self.conn.execute(
            "SELECT * FROM grades WHERE student_id=? ORDER BY subject", (sid,)
        ).fetchall()

    def get_all_grades(self):
        return self.conn.execute(
            """SELECT g.*, s.name as student_name FROM grades g
              JOIN students s ON g.student_id = s.id
              ORDER BY s.id, g.subject"""
        ).fetchall()

    def upsert_grade(self, sid, subject, marks, max_marks=100):
        self.conn.execute(
            """INSERT INTO grades (student_id,subject,marks,max_marks)
              VALUES (?,?,?,?)
              ON CONFLICT(student_id,subject)
              DO UPDATE SET marks=excluded.marks, max_marks=excluded.max_marks,
                            updated_at=CURRENT_TIMESTAMP""",
            (sid, subject, marks, max_marks)
        )
        self.conn.commit()

    def delete_grade(self, gid):
        self.conn.execute("DELETE FROM grades WHERE id=?", (gid,))
        self.conn.commit()

    def close(self):
```

```
        self.conn.close()


#  DATA STRUCTURES

class Node:
    def __init__(self, data): self.data = data; self.next = None

class LinkedList:
    def __init__(self): self.head = None
    def push_front(self, d):
        n = Node(d); n.next = self.head; self.head = n
    def to_list(self):
        r, c = [], self.head
        while c: r.append(c.data); c = c.next
        return r

class Stack:
    def __init__(self): self._i = []
    def push(self, x):  self._i.append(x)
    def pop(self):      return self._i.pop() if self._i else None
    def is_empty(self): return not self._i
    def to_list(self):  return list(reversed(self._i))

class Queue:
    def __init__(self): self._i = []
    def enqueue(self, x): self._i.append(x)
    def dequeue(self):    return self._i.pop(0) if self._i else None
    def is_empty(self):   return not self._i
    def size(self):       return len(self._i)


#  COLOUR PALETTES

DARK = dict(
    bg='#0d1117', surface='#161b22', card='#21262d', border='#30363d',
    accent='#58a6ff', accent2='#f78166', success='#3fb950',
    warning='#d29922', danger='#f85149', text='#c9d1d9',
    text_dim='#8b949e', input_bg='#0d1117', gold='#ffd700',
    admin='#8957e5',
)
LIGHT = dict(
    bg='#f6f8fa', surface='#ffffff', card='#ffffff', border='#d0d7de',
    accent='#0969da', accent2='#cf222e', success='#1a7f37',
```

```
    warning='#9a6700', danger='#cf222e', text='#1f2328',
    text_dim='#57606a', input_bg='#ffffff', gold='#bf8700',
    admin='#6639ba',
)

# Admin credentials
ADMIN_ID = "ADMIN"
ADMIN_PASSWORD = "admin123"


#  WIDGET HELPERS

class ModernButton(tk.Button):
    def __init__(self, parent, text, command, color=None, **kw):
        self._c = color or DARK['accent']
        self._h = self._dim(self._c)
        d = dict(bg=self._c, fg='white', font=('Segoe UI', 11, 'bold'),
                 relief='flat', bd=0, padx=16, pady=9, cursor='hand2',
                 activebackground=self._h, activeforeground='white')
        d.update(kw)
        super().__init__(parent, text=text, command=command, **d)
        self.bind('<Enter>', lambda e: self.config(bg=self._h))
        self.bind('<Leave>', lambda e: self.config(bg=self._c))

    @staticmethod
    def _dim(h):
        try:
            h = h.lstrip('#')
            return '#{:02x}{:02x}{:02x}'.format(
                max(0, int(h[0:2], 16) - 25),
                max(0, int(h[2:4], 16) - 25),
                max(0, int(h[4:6], 16) - 25))
        except Exception:
            return h


def _card(parent, C, **kw):
    d = dict(bg=C['card'], relief='flat', bd=0, padx=10, pady=6)
    d.update(kw)
    return tk.Frame(parent, **d)


def _entry(parent, C, **kw):
    d = dict(bg=C['input_bg'], fg=C['text'], insertbackground=C['text'],
```

```
            relief='solid', bd=1, font=('Segoe UI', 11))
    d.update(kw)
    return tk.Entry(parent, **d)


def _sep(parent, C):
    return tk.Frame(parent, bg=C['border'], height=1)

#  MAIN APPLICATION

class CampusConnect:

    TIMETABLE = [
        ["07:30-08:30",
         "",
         "Hindi Bhasha\n(Asst. Prof. Mukesh Verma)\nR.209",
         "Descriptive Statistics\n(Asst. Prof. Chetan Kanojia)\nR.003",
         "(CC) Intro to Sports\nPhysical Literacy\nHealth & Fitness & Yog\nR.209",
         "Env. Mgmt & Sustainable Dev-I\n(Asst. Prof. Chetana Tanavade)\nR.209",
         "Adv. Python Prog. Practical\nBatch-2\n(Asst. Prof. Rohit Sahu)\nLab 01"],
        ["08:30-09:30",
         "Content Writing\n(Asst. Prof. Jyoti Chauhan)\nR.003",
         "", "", "", "", ""],
        ["09:30-10:30",
         "Object Oriented Programming\n(Asst. Prof. Jyoti Chauhan)\nR.003",
         "Web Technologies\n(Asst. Prof. Pradnya Kharade)\nR.209",
         "Basic Concepts in Research\n(Dr. Mahendra Kanojia)\nR.003",
         "Basic Concepts in Research\n(Dr. Mahendra Kanojia)\nR.003",
         "Advanced Python Programming\n(Dr. Mahendra Kanojia)\nR.003",
         "Web Technologies Practical\nBatch-1\n(Asst. Prof. Pradnya Kharade)\nLab 01"],
        ["10:30-10:50",
         "— BREAK —", "— BREAK —", "— BREAK —", "— BREAK —", "— BREAK —", "—
BREAK —"],
        ["10:50-11:50",
         "Web Technologies\n(Asst. Prof. Pradnya Kharade)\nR.003",
         "Design & Analysis of Algorithms\n(Asst. Prof. Pradnya Kharade)\nR.209",
         "Advanced Python Programming\n(Dr. Mahendra Kanojia)\nR.003",
         "Design & Analysis of Algorithms\n(Asst. Prof. Pradnya Kharade)\nR.003",
         "Object Oriented Programming\n(Asst. Prof. Jyoti Chauhan)\nR.209",
         "Web Technologies Practical\nBatch-1\n(Asst. Prof. Pradnya Kharade)\nLab 01"],
        ["11:50-12:50",
         "Web Technologies Practical\nBatch-2\n(Asst. Prof. Pradnya Kharade)\nLab 01",
         "Adv. Python Prog. Practical\nBatch-1\n(Asst. Prof. Rohit Sahu)\nLab 01",
         "OOP Practical Batch-2\n(Asst. Prof. Jyoti Chauhan)\nLab 01",
```

```
    "OOP Practical Batch-1\n(Asst. Prof. Jyoti Chauhan)\nLab 01",
    "Content Writing\n(Asst. Prof. Jyoti Chauhan)\nR.209",
    ""],
   ["12:50-01:50", "", "", "", "", "Library", ""],
   ["01:50-02:50",
    "DAA Practical Batch-2\n(Asst. Prof. Pradnya Kharade)\nLab 01",
    "DAA Practical Batch-1\n(Asst. Prof. Pradnya Kharade)\nLab 01",
    "", "", "", ""],
   ["02:50-03:50", "", "", "", "", "", ""],
]

DAYS = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"]

SUBJECTS = [
   "Web Technologies",
   "Object Oriented Programming",
   "Advanced Python Programming",
   "Design & Analysis of Algorithms",
   "Basic Concepts in Research",
   "Descriptive Statistics",
   "Content Writing",
   "Hindi Bhasha - Kaushal Ke Adhar",
   "Environmental Mgmt & Sustainable Development - I",
]

LAB_SOFTWARE = [
   "VS Code + Python 3.x",
   "Eclipse IDE (Java)",
   "NetBeans IDE",
   "PyCharm Community Edition",
   "XAMPP (Apache + MySQL + PHP)",
   "Visual Studio Code + Live Server",
   "Jupyter Notebook",
   "Sublime Text",
   "MySQL Workbench",
   "Postman (API Testing)",
   "Git + GitHub Desktop",
   "Google Chrome (Web Dev Tools)",
]

# Init
def __init__(self, root):
   self.root = root
   self.root.title("Campus Connect — FYCS Semester I | A.Y. 2025-26")
```

```
    self.root.geometry("1440x880")
    self.root.minsize(1100, 700)
    self.C = DARK
    self.dark_mode = True
    self.current_user = None
    self.is_admin = False
    self.db = DatabaseManager()
    self._style()
    self._build_login()

def _style(self):
    C = self.C
    s = ttk.Style()
    try: s.theme_use('clam')
    except Exception: pass
    s.configure('TNotebook', background=C['surface'], borderwidth=0)
    s.configure('TNotebook.Tab', background=C['card'], foreground=C['text_dim'],
                padding=[18, 10], font=('Segoe UI', 11, 'bold'), borderwidth=0)
    s.map('TNotebook.Tab',
          background=[('selected', C['accent'])],
          foreground=[('selected', '#ffffff')])
    s.configure('Treeview', background=C['card'], foreground=C['text'],
                fieldbackground=C['card'], rowheight=30, font=('Segoe UI', 10))
    s.configure('Treeview.Heading', background=C['surface'],
                foreground=C['accent'], font=('Segoe UI', 11, 'bold'))
    s.map('Treeview', background=[('selected', C['accent'])])

def _clear(self):
    for w in self.root.winfo_children(): w.destroy()

    #  LOGIN
    def _build_login(self):
    self._clear()
    C = self.C
    self.root.configure(bg=C['bg'])

    outer = tk.Frame(self.root, bg=C['bg'])
    outer.place(relx=.5, rely=.5, anchor='center')

    box = tk.Frame(outer, bg=C['surface'])
    box.pack(ipadx=55, ipady=35)

    tk.Label(box, text="🎓", bg=C['surface'], font=('Segoe UI', 52)).pack(pady=(30, 0))
    tk.Label(box, text="Campus Connect", bg=C['surface'], fg=C['accent'],
```

```
        font=('Segoe UI', 28, 'bold')).pack()
tk.Label(box, text="First Year Computer Science  |  Semester I",
        bg=C['surface'], fg=C['text_dim'], font=('Segoe UI', 12)).pack(pady=(3, 2))
tk.Label(box, text="Sheth L.U.J. College  •  A.Y. 2025-26",
        bg=C['surface'], fg=C['text_dim'], font=('Segoe UI', 10)).pack(pady=(0, 18))

_sep(box, C).pack(fill='x', padx=40, pady=4)

# ── Student login ──
tk.Label(box, text="👤  Student Login", bg=C['surface'], fg=C['accent'],
        font=('Segoe UI', 12, 'bold')).pack(pady=(14, 2))

form = tk.Frame(box, bg=C['surface'])
form.pack(pady=8, padx=64)

tk.Label(form, text="Student ID", bg=C['surface'], fg=C['text_dim'],
        font=('Segoe UI', 11)).pack(anchor='w')
self._lid = _entry(form, C, width=30)
self._lid.pack(pady=(4, 4), ipady=7)
self._lid.focus()
self._lid.bind('<Return>', lambda e: self._do_login())

tk.Label(form, text="Demo accounts:  CS001 – CS008   (any password)",
        bg=C['surface'], fg=C['text_dim'], font=('Segoe UI', 9)).pack(anchor='w', pady=(0, 10))

ModernButton(form, "  Student Login  →", self._do_login,
            color=C['accent'], font=('Segoe UI', 13, 'bold'),
            padx=24, pady=11).pack(fill='x', pady=(0, 4))
ModernButton(form, "New Student? Register", self._build_register,
            color=C['success'], font=('Segoe UI', 10), pady=7).pack(fill='x', pady=2)

_sep(box, C).pack(fill='x', padx=40, pady=12)

# ── Admin login ──
tk.Label(box, text="🛡️  Admin Login", bg=C['surface'], fg=C['admin'],
        font=('Segoe UI', 12, 'bold')).pack(pady=(0, 6))

af = tk.Frame(box, bg=C['surface'])
af.pack(padx=64, pady=(0, 6))

pw_row = tk.Frame(af, bg=C['surface'])
pw_row.pack(fill='x')
tk.Label(pw_row, text="Password:", bg=C['surface'], fg=C['text_dim'],
        font=('Segoe UI', 11)).pack(anchor='w')
```

```
        self._apw = _entry(pw_row, C, width=30, show='●')
        self._apw.pack(pady=(4, 8), ipady=7)
        self._apw.bind('<Return>', lambda e: self._do_admin_login())

        tk.Label(af, text="Admin ID: ADMIN  |  Password: admin123",
                 bg=C['surface'], fg=C['text_dim'], font=('Segoe UI', 9)).pack(anchor='w', pady=(0, 8))
        ModernButton(af, "🛡️  Admin Login  →", self._do_admin_login,
                     color=C['admin'], font=('Segoe UI', 12, 'bold'),
                     padx=24, pady=11).pack(fill='x')

        tk.Button(self.root, text="☀ / 🌙  Toggle Theme",
                  bg=C['surface'], fg=C['text_dim'], font=('Segoe UI', 9),
                  relief='flat', cursor='hand2',
                  command=self._toggle_theme).pack(side='bottom', pady=10)

    def _do_login(self):
        sid = self._lid.get().strip().upper()
        row = self.db.get_student(sid)
        if row:
            self.current_user = dict(row)
            self.is_admin = False
            self._build_dashboard()
        else:
            messagebox.showerror("Login Failed",
                                 "Student ID not found.\nDemo accounts: CS001 – CS008")

    def _do_admin_login(self):
        pw = self._apw.get().strip()
        if pw == ADMIN_PASSWORD:
            self.current_user = {'id': ADMIN_ID, 'name': 'Administrator', 'roll_no': 'N/A'}
            self.is_admin = True
            self._build_admin_dashboard()
        else:
            messagebox.showerror("Admin Login Failed", "Incorrect admin password.\nDefault:
admin123")

    #  REGISTER
    def _build_register(self):
        self._clear()
        C = self.C
        self.root.configure(bg=C['bg'])

        box = tk.Frame(self.root, bg=C['surface'])
        box.place(relx=.5, rely=.5, anchor='center')
```

```
        tk.Label(box, text="New Student Registration", bg=C['surface'], fg=C['accent'],
                font=('Segoe UI', 20, 'bold')).pack(pady=(28, 20), padx=60)

        form = tk.Frame(box, bg=C['surface'])
        form.pack(padx=60, pady=6)

        fields = [
            ("Student ID *",          'sid'),
            ("Full Name *",           'name'),
            ("Roll Number",           'roll'),
            ("Phone Number",          'phone'),
            ("Year (e.g. FY)",        'year'),
            ("Department (e.g. CS)",  'dept'),
        ]
        self._rv = {}
        for lbl, key in fields:
            tk.Label(form, text=lbl, bg=C['surface'], fg=C['text_dim'],
                    font=('Segoe UI', 11), anchor='w').pack(fill='x', pady=(8, 0))
            e = _entry(form, C, width=36)
            e.pack(ipady=5, pady=(2, 0), fill='x')
            self._rv[key] = e

        self._rv['year'].insert(0, 'FY')
        self._rv['dept'].insert(0, 'CS')

        btns = tk.Frame(box, bg=C['surface'])
        btns.pack(pady=20, padx=60, fill='x')
        ModernButton(btns, "Register", self._do_register, color=C['success']).pack(side='left',
expand=True, padx=4)
        ModernButton(btns, "Back",     self._build_login, color=C['danger']).pack(side='right',
expand=True, padx=4)

    def _do_register(self):
        v = {k: e.get().strip() for k, e in self._rv.items()}
        sid = v['sid'].upper()
        if not v['sid'] or not v['name']:
            messagebox.showerror("Error", "Student ID and Name are required."); return
        ok, msg = self.db.add_student(sid, v['name'], v['phone'],
                                      v['year'] or 'FY', v['dept'] or 'CS', v['roll'])
        if ok:
            messagebox.showinfo("Success", f"Welcome, {v['name']}!\n{msg}")
            self._build_login()
        else:
```

```
        messagebox.showerror("Error", msg)

# ═══ SHARED TOPBAR HELPER ═══
def _build_topbar(self, label_text, badge_color):
    C = self.C
    bar = tk.Frame(self.root, bg=C['surface'], height=58)
    bar.pack(fill='x')
    bar.pack_propagate(False)

    tk.Label(bar, text="🎓  Campus Connect", bg=C['surface'], fg=C['accent'],
             font=('Segoe UI', 17, 'bold')).pack(side='left', padx=18)
    tk.Label(bar, text=label_text,
             bg=C['surface'], fg=C['text_dim'], font=('Segoe UI', 10)).pack(side='left')

    ctrl = tk.Frame(bar, bg=C['surface'])
    ctrl.pack(side='right', padx=14)

    u = self.current_user
    badge = tk.Label(ctrl,
                     text=f"  {'🛡️ Admin' if self.is_admin else '👤 ' + u['name']}  ({u['id']})  ",
                     bg=badge_color, fg='white', font=('Segoe UI', 10, 'bold'),
                     relief='flat', padx=6)
    badge.pack(side='left', padx=10, ipady=3)

    ModernButton(ctrl, "☀/🌙", self._toggle_theme, color=C['card'],
                 fg=C['text'], font=('Segoe UI', 10), padx=8, pady=5).pack(side='left', padx=3)
    ModernButton(ctrl, "Logout", self._logout, color=C['danger'],
                 font=('Segoe UI', 10), padx=8, pady=5).pack(side='left', padx=3)

    _sep(self.root, C).pack(fill='x')

# ═══ STUDENT DASHBOARD ═══
def _build_dashboard(self):
    self._clear()
    C = self.C
    self.root.configure(bg=C['bg'])
    self._build_topbar("FYCS  •  Semester I  •  A.Y. 2025-26", C['accent'])

    nb = ttk.Notebook(self.root)
    nb.pack(fill='both', expand=True)

    for name, fn in [
        ("📢  Announcements", self._tab_ann),
        ("📚  Assignments",   self._tab_asgn),
```

```
        ("💻 Lab Booking",   self._tab_lab),
        ("📅 Timetable",     self._tab_tt),
        ("📊 Grades",        self._tab_grades),
        ("👥 Directory",     self._tab_dir),
    ]:
        frm = tk.Frame(nb, bg=C['bg'])
        nb.add(frm, text=name)
        fn(frm)

# ADMIN DASHBOARD
def _build_admin_dashboard(self):
    self._clear()
    C = self.C
    self.root.configure(bg=C['bg'])
    self._build_topbar("Admin Panel • FYCS Sem I • A.Y. 2025-26", C['admin'])

    nb = ttk.Notebook(self.root)
    nb.pack(fill='both', expand=True)

    for name, fn in [
        ("🏠 Overview",          self._admin_tab_overview),
        ("👥 Manage Students",   self._admin_tab_students),
        ("📚 All Assignments",   self._admin_tab_assignments),
        ("📢 Announcements",     self._admin_tab_announcements),
        ("💻 Lab Bookings",      self._admin_tab_lab),
        ("📊 All Grades",        self._admin_tab_grades),
    ]:
        frm = tk.Frame(nb, bg=C['bg'])
        nb.add(frm, text=name)
        fn(frm)

# ADMIN TAB: OVERVIEW
def _admin_tab_overview(self, p):
    C = self.C

    tk.Label(p, text="🛡️ Admin Overview — FYCS Semester I",
             bg=C['bg'], fg=C['admin'], font=('Segoe UI', 18, 'bold')).pack(pady=(24, 4))
    tk.Label(p, text="Sheth L.U.J. College • A.Y. 2025-26",
             bg=C['bg'], fg=C['text_dim'], font=('Segoe UI', 11)).pack(pady=(0, 24))

    cards_frame = tk.Frame(p, bg=C['bg'])
    cards_frame.pack(pady=10)

    stats = [
```

```
        ("👥", "Total Students",    str(self.db.count_students()),         C['accent']),
        ("📚", "Total Submissions", str(self.db.count_assignments()),       C['success']),
        ("📢", "Announcements",     str(self.db.count_announcements()),     C['warning']),
        ("💻", "Lab Queue",         str(self.db.count_waiting_bookings()), C['danger']),
    ]

    for icon, label, value, color in stats:
        card = tk.Frame(cards_frame, bg=C['card'], relief='flat', bd=0)
        card.pack(side='left', padx=16, ipadx=30, ipady=20)
        tk.Label(card, text=icon,   bg=C['card'], font=('Segoe UI', 32)).pack()
        tk.Label(card, text=value,  bg=C['card'], fg=color,       font=('Segoe UI', 28, 'bold')).pack()
        tk.Label(card, text=label,  bg=C['card'], fg=C['text_dim'], font=('Segoe UI', 11)).pack()

    _sep(p, C).pack(fill='x', padx=40, pady=24)

    # Quick actions
    tk.Label(p, text="⚡  Quick Actions", bg=C['bg'], fg=C['text'],
            font=('Segoe UI', 14, 'bold')).pack(pady=(0, 12))

    qa = tk.Frame(p, bg=C['bg'])
    qa.pack()
    ModernButton(qa, "➕  Add New Student", self._admin_add_student_dialog,
                color=C['accent'], font=('Segoe UI', 11, 'bold'), padx=20, pady=10).pack(side='left',
padx=8)
    ModernButton(qa, "📢  Post Announcement", self._admin_quick_announcement,
                color=C['warning'], font=('Segoe UI', 11, 'bold'), padx=20, pady=10).pack(side='left',
padx=8)
    ModernButton(qa, "⚙  Process Lab Queue", self._admin_process_lab,
                color=C['success'], font=('Segoe UI', 11, 'bold'), padx=20,
pady=10).pack(side='left', padx=8)

    _sep(p, C).pack(fill='x', padx=40, pady=24)

    tk.Label(p, text=f"  Logged in as: Administrator  |  Session:
{datetime.now().strftime('%d-%b-%Y  %H:%M')}",
            bg=C['bg'], fg=C['text_dim'], font=('Segoe UI', 10)).pack(anchor='w', padx=40)

  def _admin_quick_announcement(self):
    C = self.C
    win = tk.Toplevel(self.root)
    win.title("Post Announcement")
    win.configure(bg=C['surface'])
    win.resizable(False, False)
```

```
        tk.Label(win, text="📢 Post Announcement", bg=C['surface'], fg=C['accent'],
                font=('Segoe UI', 14, 'bold')).pack(padx=30, pady=(20, 10))
        msg_e = tk.Text(win, width=50, height=5, bg=C['input_bg'], fg=C['text'],
                        font=('Segoe UI', 11), relief='solid', bd=1, insertbackground=C['text'])
        msg_e.pack(padx=30, pady=8)

        def post():
            msg = msg_e.get('1.0', tk.END).strip()
            if not msg: messagebox.showerror("Error", "Message cannot be empty."); return
            self.db.add_announcement(msg, "Admin")
            win.destroy()
            messagebox.showinfo("Posted", "Announcement posted successfully!")

        ModernButton(win, "Post", post, color=C['accent']).pack(padx=30, pady=(4, 6), fill='x')
        ModernButton(win, "Cancel", win.destroy, color=C['danger']).pack(padx=30, pady=(0, 16),
fill='x')

    def _admin_process_lab(self):
        item = self.db.process_next_booking()
        if item:
            messagebox.showinfo("Processed",
                                f"{item['student_name']}\nSoftware: {item['software']}\n"
                                f"Purpose: {item['purpose']}\n\nLab seat is now available!")
        else:
            messagebox.showinfo("Empty", "No bookings in queue.")

    #  ADMIN TAB: MANAGE STUDENTS
    def _admin_tab_students(self, p):
        C = self.C

        hf = tk.Frame(p, bg=C['bg'])
        hf.pack(fill='x', padx=20, pady=12)
        tk.Label(hf, text="👥  Student Management", bg=C['bg'], fg=C['admin'],
                font=('Segoe UI', 17, 'bold')).pack(side='left')

        bar = _card(p, C)
        bar.pack(fill='x', padx=20, pady=(0, 10))

        tk.Label(bar, text="Search:", bg=C['card'], fg=C['text_dim'],
                font=('Segoe UI', 11)).pack(side='left', padx=8)
        self._ads = tk.StringVar()
        se = _entry(bar, C, textvariable=self._ads, width=30)
        se.pack(side='left', padx=6, ipady=5)
        se.bind('<KeyRelease>', lambda e: self._admin_ref_students())
```

```
        ModernButton(bar, "➕ Add Student", self._admin_add_student_dialog,
                     color=C['success'], font=('Segoe UI', 10), pady=6).pack(side='left', padx=10)
        ModernButton(bar, "✏ Edit", self._admin_edit_student,
                     color=C['warning'], font=('Segoe UI', 10), pady=6).pack(side='right', padx=4)
        ModernButton(bar, "🗑️ Delete", self._admin_del_student,
                     color=C['danger'], font=('Segoe UI', 10), pady=6).pack(side='right', padx=4)

        self._adrt = ttk.Treeview(p,
                                  columns=('id', 'roll', 'name', 'phone', 'year', 'dept', 'created'),
                                  show='headings', height=22)
        for col, txt, w in [('id', 'Student ID', 90), ('roll', 'Roll No', 75),
                            ('name', 'Full Name', 200), ('phone', 'Phone', 130),
                            ('year', 'Year', 55), ('dept', 'Dept', 55),
                            ('created', 'Registered', 150)]:
            self._adrt.heading(col, text=txt)
            self._adrt.column(col, width=w)

        vsb = ttk.Scrollbar(p, orient='vertical', command=self._adrt.yview)
        self._adrt.configure(yscrollcommand=vsb.set)
        vsb.pack(side='right', fill='y', padx=(0, 8))
        self._adrt.pack(fill='both', expand=True, padx=20, pady=4)

        self._adc = tk.Label(p, text="", bg=C['bg'], fg=C['text_dim'], font=('Segoe UI', 10))
        self._adc.pack(anchor='e', padx=24, pady=4)
        self._admin_ref_students()

    def _admin_ref_students(self):
        q = self._ads.get().strip() if hasattr(self, '_ads') else ''
        for r in self._adrt.get_children(): self._adrt.delete(r)
        rows = self.db.search_students(q) if q else self.db.get_all_students()
        for r in rows:
            self._adrt.insert('', 'end',
                              values=(r['id'], r['roll_no'], r['name'], r['phone'],
                                      r['year'], r['department'], r['created_at']))
        self._adc.config(text=f"{len(rows)} student(s)")

    def _admin_del_student(self):
        sel = self._adrt.selection()
        if not sel: messagebox.showinfo("Info", "Select a student to delete."); return
        sid = self._adrt.item(sel[0])['values'][0]
        if messagebox.askyesno("Confirm Delete", f"Delete student {sid}? This cannot be
undone."):
            self.db.delete_student(sid)
```

```
        self._admin_ref_students()
        messagebox.showinfo("Deleted", f"Student {sid} deleted.")

    def _admin_edit_student(self):
        sel = self._adrt.selection()
        if not sel: messagebox.showinfo("Info", "Select a student to edit."); return
        sid = self._adrt.item(sel[0])['values'][0]
        row = self.db.get_student(sid)
        if not row: return
        self._open_edit_student_window(sid, row, self._admin_ref_students)

    def _admin_add_student_dialog(self):
        C = self.C
        win = tk.Toplevel(self.root)
        win.title("Add New Student")
        win.configure(bg=C['surface'])
        win.resizable(False, False)

        tk.Label(win, text="➕  Add New Student", bg=C['surface'], fg=C['accent'],
                font=('Segoe UI', 14, 'bold')).pack(padx=40, pady=(20, 10))

        form = tk.Frame(win, bg=C['surface'])
        form.pack(padx=40, pady=6)

        fields = [("Student ID *", 'sid'), ("Full Name *", 'name'),
                  ("Roll Number", 'roll'), ("Phone", 'phone'),
                  ("Year", 'year'), ("Department", 'dept')]
        ents = {}
        for lbl, key in fields:
            tk.Label(form, text=lbl + ":", bg=C['surface'], fg=C['text_dim'],
                     font=('Segoe UI', 11)).pack(anchor='w')
            e = _entry(form, C, width=34)
            e.pack(pady=(2, 8), ipady=4, fill='x')
            ents[key] = e
        ents['year'].insert(0, 'FY')
        ents['dept'].insert(0, 'CS')

        def add():
            sid = ents['sid'].get().strip().upper()
            name = ents['name'].get().strip()
            if not sid or not name:
                messagebox.showerror("Error", "ID and Name are required."); return
            ok, msg = self.db.add_student(sid, name, ents['phone'].get().strip(),
                                          ents['year'].get().strip() or 'FY',
```

```
                                ents['dept'].get().strip() or 'CS',
                                ents['roll'].get().strip())
            if ok:
                win.destroy()
                if hasattr(self, '_adrt'): self._admin_ref_students()
                messagebox.showinfo("Added", f"Student {sid} added successfully!")
            else:
                messagebox.showerror("Error", msg)

        ModernButton(win, "Add Student", add, color=C['success']).pack(padx=40, pady=(4, 6),
fill='x')
        ModernButton(win, "Cancel", win.destroy, color=C['danger']).pack(padx=40, pady=(0, 16),
fill='x')

    #  ADMIN TAB: ALL ASSIGNMENTS
    def _admin_tab_assignments(self, p):
        C = self.C

        hf = tk.Frame(p, bg=C['bg'])
        hf.pack(fill='x', padx=20, pady=12)
        tk.Label(hf, text="📚  All Assignments — All Students", bg=C['bg'], fg=C['admin'],
                font=('Segoe UI', 17, 'bold')).pack(side='left')

        bar = _card(p, C)
        bar.pack(fill='x', padx=20, pady=(0, 10))
        ModernButton(bar, "🔄 Refresh", self._admin_ref_assignments,
                    color=C['accent'], font=('Segoe UI', 10), pady=6).pack(side='left', padx=8)
        ModernButton(bar, "🗑️ Delete Selected", self._admin_del_assignment,
                    color=C['danger'], font=('Segoe UI', 10), pady=6).pack(side='right', padx=8)

        self._aasgt = ttk.Treeview(p,
                                   columns=('id', 'student_id', 'student', 'subject', 'title', 'file', 'when'),
                                   show='headings', height=24)
        for col, txt, w in [('id', '#', 40), ('student_id', 'ID', 70),
                            ('student', 'Student', 160), ('subject', 'Subject', 180),
                            ('title', 'Title', 190), ('file', 'Filename', 150),
                            ('when', 'Submitted At', 140)]:
            self._aasgt.heading(col, text=txt)
            self._aasgt.column(col, width=w)

        vsb = ttk.Scrollbar(p, orient='vertical', command=self._aasgt.yview)
        self._aasgt.configure(yscrollcommand=vsb.set)
        vsb.pack(side='right', fill='y', padx=(0, 8))
        self._aasgt.pack(fill='both', expand=True, padx=20, pady=4)
```

```
    self._aasgl = tk.Label(p, text="", bg=C['bg'], fg=C['text_dim'], font=('Segoe UI', 10))
    self._aasgl.pack(anchor='e', padx=24, pady=4)
    self._admin_ref_assignments()

def _admin_ref_assignments(self):
    if not hasattr(self, '_aasgt'): return
    for r in self._aasgt.get_children(): self._aasgt.delete(r)
    rows = self.db.get_all_assignments()
    for i, r in enumerate(rows, 1):
        self._aasgt.insert('', 'end',
                           values=(r['id'], r['student_id'], r['student_name'],
                                   r['subject'], r['title'], r['filename'], r['submitted_at']))
    self._aasgl.config(text=f"{len(rows)} total submission(s)")

def _admin_del_assignment(self):
    sel = self._aasgt.selection()
    if not sel: messagebox.showinfo("Info", "Select an assignment to delete."); return
    aid = self._aasgt.item(sel[0])['values'][0]
    title = self._aasgt.item(sel[0])['values'][4]
    if messagebox.askyesno("Confirm Delete", f"Delete assignment:\n\"{title}\"?"):
        self.db.delete_assignment_by_id(aid)
        self._admin_ref_assignments()
        messagebox.showinfo("Deleted", "Assignment deleted.")

#  ADMIN TAB: ANNOUNCEMENTS
def _admin_tab_announcements(self, p):
    C = self.C

    tk.Label(p, text="📢 Announcements Management", bg=C['bg'], fg=C['admin'],
             font=('Segoe UI', 17, 'bold')).pack(anchor='w', padx=20, pady=(14, 6))

    pb = _card(p, C)
    pb.pack(fill='x', padx=20, pady=(0, 10))
    tk.Label(pb, text="New Message:", bg=C['card'], fg=C['text_dim'],
             font=('Segoe UI', 11)).pack(side='left', padx=6)
    self._aae = _entry(pb, C, width=62)
    self._aae.pack(side='left', padx=6, ipady=5)
    ModernButton(pb, "📢 Post", self._admin_post_ann, color=C['accent'],
                 font=('Segoe UI', 10), padx=12, pady=6).pack(side='left', padx=4)
    ModernButton(pb, "🗑️ Delete Selected", self._admin_del_ann, color=C['danger'],
                 font=('Segoe UI', 10), pady=6).pack(side='right', padx=6)

    self._aat = ttk.Treeview(p, columns=('id', 'date', 'author', 'message'),
```

```
                        show='headings', height=22)
    for col, txt, w in [('id', '#', 45), ('date', 'Date & Time', 155),
                        ('author', 'Author', 130), ('message', 'Message', 770)]:
        self._aat.heading(col, text=txt)
        self._aat.column(col, width=w)
    vsb = ttk.Scrollbar(p, orient='vertical', command=self._aat.yview)
    self._aat.configure(yscrollcommand=vsb.set)
    vsb.pack(side='right', fill='y', padx=(0, 8))
    self._aat.pack(fill='both', expand=True, padx=20, pady=4)
    self._admin_ref_ann()

def _admin_post_ann(self):
    msg = self._aae.get().strip()
    if not msg: messagebox.showerror("Error", "Please type a message."); return
    self.db.add_announcement(msg, "Admin")
    self._aae.delete(0, tk.END)
    self._admin_ref_ann()

def _admin_del_ann(self):
    sel = self._aat.selection()
    if not sel: messagebox.showinfo("Info", "Select an announcement to delete."); return
    aid = self._aat.item(sel[0])['values'][0]
    if messagebox.askyesno("Delete", "Delete this announcement?"):
        self.db.delete_announcement(aid)
        self._admin_ref_ann()

def _admin_ref_ann(self):
    if not hasattr(self, '_aat'): return
    for r in self._aat.get_children(): self._aat.delete(r)
    for r in self.db.get_announcements():
        self._aat.insert('', 'end', values=(r['id'], r['created_at'], r['author'], r['message']))

#  ADMIN TAB: LAB BOOKINGS
def _admin_tab_lab(self, p):
    C = self.C

    hf = tk.Frame(p, bg=C['bg'])
    hf.pack(fill='x', padx=20, pady=12)
    tk.Label(hf, text="💻  Lab Booking Management", bg=C['bg'], fg=C['admin'],
             font=('Segoe UI', 17, 'bold')).pack(side='left')

    bar = _card(p, C)
    bar.pack(fill='x', padx=20, pady=(0, 10))
    ModernButton(bar, "⚙ Process Next (Waiting)", self._admin_process_next_lab,
```

```
            color=C['success'], font=('Segoe UI', 10), pady=6).pack(side='left', padx=8)
        ModernButton(bar, "🗑️ Delete Selected", self._admin_del_booking,
            color=C['danger'], font=('Segoe UI', 10), pady=6).pack(side='right', padx=8)
        ModernButton(bar, "🔄 Refresh", self._admin_ref_lab,
            color=C['accent'], font=('Segoe UI', 10), pady=6).pack(side='right', padx=4)

        self._albt = ttk.Treeview(p,
                    columns=('id', 'student', 'software', 'purpose', 'hours', 'status', 'booked_at'),
                    show='headings', height=24)
        for col, txt, w in [('id', '#', 40), ('student', 'Student', 160),
                ('software', 'Software', 200), ('purpose', 'Purpose', 220),
                ('hours', 'Hrs', 50), ('status', 'Status', 90),
                ('booked_at', 'Booked At', 140)]:
            self._albt.heading(col, text=txt)
            self._albt.column(col, width=w)
        self._albt.tag_configure('waiting',   foreground=C['warning'])
        self._albt.tag_configure('completed', foreground=C['success'])

        vsb = ttk.Scrollbar(p, orient='vertical', command=self._albt.yview)
        self._albt.configure(yscrollcommand=vsb.set)
        vsb.pack(side='right', fill='y', padx=(0, 8))
        self._albt.pack(fill='both', expand=True, padx=20, pady=4)
        self._admin_ref_lab()

    def _admin_ref_lab(self):
        if not hasattr(self, '_albt'): return
        for r in self._albt.get_children(): self._albt.delete(r)
        for r in self.db.get_all_bookings():
            tag = (r['status'],)
            self._albt.insert('', 'end', tags=tag,
                    values=(r['id'], r['student_name'], r['software'],
                        r['purpose'], r['duration'], r['status'].upper(), r['booked_at']))

    def _admin_process_next_lab(self):
        item = self.db.process_next_booking()
        if item:
            self._admin_ref_lab()
            messagebox.showinfo("Processed", f"{item['student_name']}\n{item['software']}\nMarked
as completed.")
        else:
            messagebox.showinfo("Empty", "No waiting bookings.")

    def _admin_del_booking(self):
        sel = self._albt.selection()
```

```
        if not sel: messagebox.showinfo("Info", "Select a booking to delete."); return
        bid = self._albt.item(sel[0])['values'][0]
        if messagebox.askyesno("Confirm Delete", f"Delete booking #{bid}?"):
            self.db.delete_booking(bid)
            self._admin_ref_lab()

    #  ADMIN TAB: ALL GRADES
    def _admin_tab_grades(self, p):
        C = self.C

        hf = tk.Frame(p, bg=C['bg'])
        hf.pack(fill='x', padx=20, pady=12)
        tk.Label(hf, text="📊  Grade Management — All Students", bg=C['bg'], fg=C['admin'],
                font=('Segoe UI', 17, 'bold')).pack(side='left')

        # Input bar for admin to add/update grade for any student
        ib = _card(p, C)
        ib.pack(fill='x', padx=20, pady=(0, 10))

        tk.Label(ib, text="Student ID:", bg=C['card'], fg=C['text_dim'],
                font=('Segoe UI', 11)).pack(side='left', padx=6)
        self._agsid = _entry(ib, C, width=8)
        self._agsid.pack(side='left', padx=4, ipady=4)

        tk.Label(ib, text="Subject:", bg=C['card'], fg=C['text_dim'],
                font=('Segoe UI', 11)).pack(side='left', padx=6)
        self._agsj = tk.StringVar(value=self.SUBJECTS[0])
        ttk.Combobox(ib, textvariable=self._agsj, values=self.SUBJECTS,
                    state='readonly', font=('Segoe UI', 10), width=28).pack(side='left', padx=4)

        tk.Label(ib, text="Marks:", bg=C['card'], fg=C['text_dim'],
                font=('Segoe UI', 11)).pack(side='left', padx=6)
        self._agm = _entry(ib, C, width=6)
        self._agm.pack(side='left', padx=4, ipady=4)

        tk.Label(ib, text="/", bg=C['card'], fg=C['text_dim'], font=('Segoe UI', 11)).pack(side='left')
        self._agmx = _entry(ib, C, width=6)
        self._agmx.insert(0, '100')
        self._agmx.pack(side='left', padx=4, ipady=4)

        ModernButton(ib, "💾 Save", self._admin_save_grade,
                    color=C['success'], font=('Segoe UI', 10), padx=12, pady=6).pack(side='left',
padx=10)
        ModernButton(ib, "🗑️ Delete Selected", self._admin_del_grade,
```

```
                color=C['danger'], font=('Segoe UI', 10), pady=6).pack(side='right', padx=8)

        self._agrt = ttk.Treeview(p,
                                  columns=('id', 'student_id', 'student', 'subject', 'marks', 'max', 'pct', 'grade',
'updated'),
                                  show='headings', height=22)
        for col, txt, w in [('id', '#', 40), ('student_id', 'ID', 70),
                            ('student', 'Student', 155), ('subject', 'Subject', 200),
                            ('marks', 'Marks', 65), ('max', 'Out of', 60),
                            ('pct', '%', 60), ('grade', 'Grade', 60),
                            ('updated', 'Updated', 130)]:
            self._agrt.heading(col, text=txt)
            self._agrt.column(col, width=w, anchor='center')

        vsb = ttk.Scrollbar(p, orient='vertical', command=self._agrt.yview)
        self._agrt.configure(yscrollcommand=vsb.set)
        vsb.pack(side='right', fill='y', padx=(0, 8))
        self._agrt.pack(fill='both', expand=True, padx=20, pady=4)
        self._admin_ref_grades()

    def _admin_ref_grades(self):
        if not hasattr(self, '_agrt'): return
        for r in self._agrt.get_children(): self._agrt.delete(r)
        for r in self.db.get_all_grades():
            pct = (r['marks'] / r['max_marks']) * 100 if r['max_marks'] else 0
            lg  = self._letter(pct)
            self._agrt.insert('', 'end',
                              values=(r['id'], r['student_id'], r['student_name'],
                                      r['subject'], f"{r['marks']:.1f}",
                                      f"{r['max_marks']:.0f}", f"{pct:.1f}%",
                                      lg, r['updated_at']))

    def _admin_save_grade(self):
        sid  = self._agsid.get().strip().upper()
        subj = self._agsj.get().strip()
        try:
            m  = float(self._agm.get().strip())
            mx = float(self._agmx.get().strip())
        except ValueError:
            messagebox.showerror("Error", "Marks must be numbers."); return
        if not sid:   messagebox.showerror("Error", "Enter a Student ID."); return
        if not subj:  messagebox.showerror("Error", "Select a subject."); return
        if not self.db.get_student(sid):
            messagebox.showerror("Error", f"Student '{sid}' not found."); return
```

```
    if not (0 <= m <= mx):
        messagebox.showerror("Error", "Marks must be between 0 and max."); return
    self.db.upsert_grade(sid, subj, m, mx)
    self._agm.delete(0, tk.END)
    self._admin_ref_grades()
    messagebox.showinfo("Saved", f"Grade saved for {sid}.")

def _admin_del_grade(self):
    sel = self._agrt.selection()
    if not sel: messagebox.showinfo("Info", "Select a grade to delete."); return
    gid = self._agrt.item(sel[0])['values'][0]
    sname = self._agrt.item(sel[0])['values'][2]
    subj  = self._agrt.item(sel[0])['values'][3]
    if messagebox.askyesno("Confirm Delete", f"Delete grade for {sname} — {subj}?"):
        self.db.delete_grade(gid)
        self._admin_ref_grades()

#  SHARED EDIT STUDENT WINDOW
def _open_edit_student_window(self, sid, row, refresh_cb):
    C = self.C
    win = tk.Toplevel(self.root)
    win.title(f"Edit Student — {sid}")
    win.configure(bg=C['surface'])
    win.resizable(False, False)

    tk.Label(win, text=f"✏  Editing: {sid}", bg=C['surface'], fg=C['accent'],
            font=('Segoe UI', 14, 'bold')).pack(padx=34, pady=(22, 12))

    form = tk.Frame(win, bg=C['surface'])
    form.pack(padx=34, pady=8)

    fields = [('Name', 'name'), ('Roll No', 'roll_no'), ('Phone', 'phone'),
             ('Year', 'year'), ('Department', 'department')]
    ents = {}
    for lbl, key in fields:
        tk.Label(form, text=lbl + ":", bg=C['surface'], fg=C['text_dim'],
                font=('Segoe UI', 11)).pack(anchor='w')
        e = _entry(form, C, width=34)
        e.insert(0, row[key] or '')
        e.pack(pady=(2, 8), ipady=4, fill='x')
        ents[key] = e

    def save():
        self.db.update_student(sid,
```

```
                        ents['name'].get().strip(),
                        ents['phone'].get().strip(),
                        ents['year'].get().strip(),
                        ents['department'].get().strip(),
                        ents['roll_no'].get().strip())
            refresh_cb()
            win.destroy()
            messagebox.showinfo("Saved", f"Student {sid} updated.")

        btns = tk.Frame(win, bg=C['surface'])
        btns.pack(pady=16, padx=34, fill='x')
        ModernButton(btns, "Save",   save,        color=C['success']).pack(side='left',  expand=True,
padx=4)
        ModernButton(btns, "Cancel", win.destroy, color=C['danger']).pack(side='right',
expand=True, padx=4)

    #  TAB: ANNOUNCEMENTS  (Student view)
    def _tab_ann(self, p):
        C = self.C

        hf = tk.Frame(p, bg=C['bg'])
        hf.pack(fill='x', padx=20, pady=14)
        tk.Label(hf, text="📢  Announcements", bg=C['bg'], fg=C['accent'],
                font=('Segoe UI', 17, 'bold')).pack(side='left')
        tk.Label(hf, text="FYCS Sem I  |  Sheth L.U.J. College",
                bg=C['bg'], fg=C['text_dim'], font=('Segoe UI', 10)).pack(side='right')

        pb = _card(p, C)
        pb.pack(fill='x', padx=20, pady=(0, 10))
        tk.Label(pb, text="Post:", bg=C['card'], fg=C['text_dim'],
                font=('Segoe UI', 11)).pack(side='left', padx=6)
        self._ae = _entry(pb, C, width=62)
        self._ae.pack(side='left', padx=6, ipady=5)
        ModernButton(pb, "Post", self._post_ann, color=C['accent'],
                font=('Segoe UI', 10), padx=12, pady=6).pack(side='left', padx=4)
        ModernButton(pb, "Delete Selected", self._del_ann, color=C['danger'],
                font=('Segoe UI', 10), pady=6).pack(side='right', padx=6)

        self._at = ttk.Treeview(p, columns=('id', 'date', 'author', 'message'),
                        show='headings', height=22)
        for col, txt, w in [('id', '#', 45), ('date', 'Date & Time', 155),
                        ('author', 'Author', 130), ('message', 'Message', 770)]:
            self._at.heading(col, text=txt)
            self._at.column(col, width=w)
```

```
    vsb = ttk.Scrollbar(p, orient='vertical', command=self._at.yview)
    self._at.configure(yscrollcommand=vsb.set)
    vsb.pack(side='right', fill='y', padx=(0, 8))
    self._at.pack(fill='both', expand=True, padx=20, pady=4)
    self._ref_ann()

def _post_ann(self):
    msg = self._ae.get().strip()
    if not msg: messagebox.showerror("Error", "Please type a message."); return
    self.db.add_announcement(msg, self.current_user['name'])
    self._ae.delete(0, tk.END)
    self._ref_ann()

def _del_ann(self):
    sel = self._at.selection()
    if not sel: messagebox.showinfo("Info", "Select an announcement to delete."); return
    aid = self._at.item(sel[0])['values'][0]
    if messagebox.askyesno("Delete", "Delete this announcement?"):
        self.db.delete_announcement(aid)
        self._ref_ann()

def _ref_ann(self):
    for r in self._at.get_children(): self._at.delete(r)
    for r in self.db.get_announcements():
        self._at.insert('', 'end', values=(r['id'], r['created_at'], r['author'], r['message']))

#  TAB: ASSIGNMENTS  (Student view — with Delete Selected)
def _tab_asgn(self, p):
    C = self.C

    left  = tk.Frame(p, bg=C['bg'])
    right = tk.Frame(p, bg=C['bg'])
    left.pack(side='left',  fill='both', expand=True, padx=(20, 8), pady=16)
    right.pack(side='right', fill='both', expand=True, padx=(8, 20), pady=16)

    # ── History ──
    tk.Label(left, text="📜  My Submission History", bg=C['bg'], fg=C['accent'],
             font=('Segoe UI', 15, 'bold')).pack(anchor='w', pady=(0, 8))

    self._sbt = ttk.Treeview(left,
                             columns=('id', 'no', 'subject', 'title', 'file', 'when'),
                             show='headings', height=18,
                             displaycolumns=('no', 'subject', 'title', 'file', 'when'))
    for col, txt, w in [('id', '#id', 0), ('no', '#', 35), ('subject', 'Subject', 175),
```

```
                    ('title', 'Title', 185), ('file', 'Filename', 150),
                    ('when', 'Submitted At', 140)]:
        self._sbt.heading(col, text=txt)
        self._sbt.column(col, width=w, anchor='center')

    vsb = ttk.Scrollbar(left, orient='vertical', command=self._sbt.yview)
    self._sbt.configure(yscrollcommand=vsb.set)
    vsb.pack(side='right', fill='y')
    self._sbt.pack(fill='both', expand=True)

    # ── Action buttons ──
    btn_frame = tk.Frame(left, bg=C['bg'])
    btn_frame.pack(fill='x', pady=8)

    ModernButton(btn_frame, "🗑️  Delete Selected", self._del_selected_asgn,
            color=C['danger'], font=('Segoe UI', 10), pady=7).pack(side='left', fill='x',
expand=True, padx=(0, 4))
    ModernButton(btn_frame, "↩  Undo Last", self._undo_asgn,
            color=C['warning'], font=('Segoe UI', 10), pady=7).pack(side='left', fill='x',
expand=True, padx=(4, 0))

    # ── New submission ──
    fc = _card(right, C)
    fc.pack(fill='both', expand=True)

    tk.Label(fc, text="New Submission", bg=C['card'], fg=C['accent'],
        font=('Segoe UI', 15, 'bold')).pack(anchor='w', padx=8, pady=(8, 14))

    tk.Label(fc, text="Subject:", bg=C['card'], fg=C['text_dim'],
        font=('Segoe UI', 11)).pack(anchor='w', padx=8)
    self._asj = tk.StringVar(value=self.SUBJECTS[0])
    ttk.Combobox(fc, textvariable=self._asj, values=self.SUBJECTS,
            state='readonly', font=('Segoe UI', 11)).pack(fill='x', padx=8, pady=(3, 12), ipady=4)

    for lbl, attr in [("Assignment Title:", '_at_'), ("Filename:", '_af_')]:
        tk.Label(fc, text=lbl, bg=C['card'], fg=C['text_dim'],
            font=('Segoe UI', 11)).pack(anchor='w', padx=8)
        e = _entry(fc, C, width=38)
        e.pack(fill='x', padx=8, pady=(3, 12), ipady=6)
        setattr(self, attr, e)

    ModernButton(fc, "✅  Submit Assignment", self._submit_asgn,
            color=C['success'], font=('Segoe UI', 12, 'bold'),
            pady=11).pack(fill='x', padx=8, pady=8)
```

```
    self._ref_asgn()

def _submit_asgn(self):
    subj  = self._asj.get().strip()
    title = self._at_.get().strip()
    fname = self._af_.get().strip()
    if not title or not fname:
        messagebox.showerror("Error", "Title and filename are required."); return
    self.db.add_assignment(self.current_user['id'], subj, title, fname)
    self._at_.delete(0, tk.END); self._af_.delete(0, tk.END)
    self._ref_asgn()
    messagebox.showinfo("Submitted", "Assignment saved to database ✔")

def _del_selected_asgn(self):
    sel = self._sbt.selection()
    if not sel:
        messagebox.showinfo("Info", "Select an assignment to delete."); return
    aid   = self._sbt.item(sel[0])['values'][0]
    title = self._sbt.item(sel[0])['values'][3]
    if messagebox.askyesno("Delete Assignment",
                           f"Delete submission:\n\"{title}\"?\n\nThis cannot be undone."):
        self.db.delete_assignment_by_id(aid)
        self._ref_asgn()
        messagebox.showinfo("Deleted", "Assignment deleted successfully.")

def _undo_asgn(self):
    ok = self.db.delete_last_assignment(self.current_user['id'])
    if ok: self._ref_asgn(); messagebox.showinfo("Undone", "Last submission removed.")
    else:  messagebox.showinfo("Info", "No submissions to undo.")

def _ref_asgn(self):
    for r in self._sbt.get_children(): self._sbt.delete(r)
    for i, r in enumerate(self.db.get_assignments(self.current_user['id']), 1):
        self._sbt.insert('', 'end',
                         values=(r['id'], i, r['subject'], r['title'],
                                 r['filename'], r['submitted_at']))

#  TAB: LAB BOOKING  (Student view)
def _tab_lab(self, p):
    C = self.C

    hf = tk.Frame(p, bg=C['bg'])
    hf.pack(fill='x', padx=20, pady=14)
```

```
        tk.Label(hf, text="💻  Computer Lab 01 — Session Booking", bg=C['bg'],
                 fg=C['accent'], font=('Segoe UI', 17, 'bold')).pack(side='left')
        tk.Label(hf, text="All Practicals in Lab 01",
                 bg=C['bg'], fg=C['warning'], font=('Segoe UI', 10, 'italic')).pack(side='right')

        ctrl = _card(p, C)
        ctrl.pack(fill='x', padx=20, pady=(0, 10))

        r1 = tk.Frame(ctrl, bg=C['card'])
        r1.pack(fill='x', pady=5)
        tk.Label(r1, text="Software / Tool:", bg=C['card'], fg=C['text_dim'],
                 font=('Segoe UI', 11)).pack(side='left', padx=8)
        self._lsw = tk.StringVar(value=self.LAB_SOFTWARE[0])
        ttk.Combobox(r1, textvariable=self._lsw, values=self.LAB_SOFTWARE,
                     state='readonly', font=('Segoe UI', 11), width=32).pack(side='left', padx=6)
        tk.Label(r1, text="Duration (hrs):", bg=C['card'], fg=C['text_dim'],
                 font=('Segoe UI', 11)).pack(side='left', padx=8)
        self._ldur = tk.StringVar(value='2')
        tk.Spinbox(r1, from_=1, to=6, textvariable=self._ldur,
                   font=('Segoe UI', 11), width=5,
                   bg=C['input_bg'], fg=C['text'], insertbackground=C['text'],
                   relief='solid', bd=1).pack(side='left', padx=6)

        r2 = tk.Frame(ctrl, bg=C['card'])
        r2.pack(fill='x', pady=5)
        tk.Label(r2, text="Purpose / Subject:", bg=C['card'], fg=C['text_dim'],
                 font=('Segoe UI', 11)).pack(side='left', padx=8)
        self._lpur = _entry(r2, C, width=46)
        self._lpur.pack(side='left', padx=6, ipady=4)

        r3 = tk.Frame(ctrl, bg=C['card'])
        r3.pack(fill='x', pady=8)
        ModernButton(r3, "Book Slot", self._book_lab,
                     color=C['success'], font=('Segoe UI', 11, 'bold'), padx=16).pack(side='left',
padx=10)

        tk.Label(p, text="Booking Queue", bg=C['bg'], fg=C['accent'],
                 font=('Segoe UI', 13, 'bold')).pack(anchor='w', padx=20, pady=(8, 4))

        self._lt = ttk.Treeview(p,
                                columns=('pos', 'student', 'software', 'purpose', 'hours', 'booked_at'),
                                show='headings', height=16)
        for col, txt, w in [('pos', '#', 40), ('student', 'Student', 170),
                            ('software', 'Software', 200), ('purpose', 'Purpose', 250),
```

```
                    ('hours', 'Hrs', 50), ('booked_at', 'Booked At', 140)]:
        self._lt.heading(col, text=txt)
        self._lt.column(col, width=w)
    vsb = ttk.Scrollbar(p, orient='vertical', command=self._lt.yview)
    self._lt.configure(yscrollcommand=vsb.set)
    vsb.pack(side='right', fill='y', padx=(0, 8))
    self._lt.pack(fill='both', expand=True, padx=20, pady=4)
    self._ref_lab()

def _book_lab(self):
    sw  = self._lsw.get()
    pur = self._lpur.get().strip()
    dur = int(self._ldur.get())
    if not pur: messagebox.showerror("Error", "Please enter a purpose/subject."); return
    u = self.current_user
    self.db.add_booking(u['id'], u['name'], sw, pur, dur)
    self._lpur.delete(0, tk.END)
    self._ref_lab()
    q = len(self.db.get_waiting_bookings())
    messagebox.showinfo("Booked!", f"{sw}\nPurpose: {pur}\nDuration: {dur} hr(s)\nQueue
position: #{q}")

def _process_lab(self):
    item = self.db.process_next_booking()
    if item:
        self._ref_lab()
        messagebox.showinfo("Processed",
                            f"{item['student_name']}\nSoftware: {item['software']}\n"
                            f"Purpose: {item['purpose']}\n\nLab seat is now available!")
    else:
        messagebox.showinfo("Empty", "No bookings in queue.")

def _ref_lab(self):
    for r in self._lt.get_children(): self._lt.delete(r)
    for i, r in enumerate(self.db.get_waiting_bookings(), 1):
        self._lt.insert('', 'end',
                        values=(i, r['student_name'], r['software'],
                                r['purpose'], r['duration'], r['booked_at']))

#  TAB: TIMETABLE
def _tab_tt(self, p):
    C = self.C

    hf = tk.Frame(p, bg=C['bg'])
```

```
        hf.pack(fill='x', padx=20, pady=12)
        tk.Label(hf, text="📅  FYCS Sem I Weekly Timetable — A.Y. 2025-26",
                 bg=C['bg'], fg=C['accent'], font=('Segoe UI', 17, 'bold')).pack(side='left')
        tk.Label(hf, text="Date: 22-11-2025",
                 bg=C['bg'], fg=C['text_dim'], font=('Segoe UI', 10)).pack(side='right')

        tk.Label(p, text="All Practicals conducted in Computer Lab 01",
                 bg=C['bg'], fg=C['warning'], font=('Segoe UI', 10, 'italic')).pack(anchor='w', padx=20,
pady=(0, 8))

        cols = ['Time'] + self.DAYS
        tree = ttk.Treeview(p, columns=cols, show='headings', height=16)
        tree.column('Time', width=110, anchor='center')
        for d in self.DAYS:
            tree.heading(d, text=d)
            tree.column(d, width=195, anchor='center')
        tree.heading('Time', text='Time')

        for row in self.TIMETABLE:
            tag = ('brk',) if '— BREAK —' in row else ()
            tree.insert('', 'end', values=row, tags=tag)
        tree.tag_configure('brk', background='#30363d', foreground=C['gold'])

        xsb = ttk.Scrollbar(p, orient='horizontal', command=tree.xview)
        vsb = ttk.Scrollbar(p, orient='vertical',   command=tree.yview)
        tree.configure(xscrollcommand=xsb.set, yscrollcommand=vsb.set)
        vsb.pack(side='right', fill='y', padx=(0, 8))
        tree.pack(fill='both', expand=True, padx=20, pady=4)
        xsb.pack(fill='x', padx=20)

        fac = tk.Frame(p, bg=C['bg'])
        fac.pack(anchor='w', padx=20, pady=6)
        tk.Label(fac, text="Faculty: ", bg=C['bg'], fg=C['text_dim'],
                 font=('Segoe UI', 9, 'bold')).pack(side='left')
        for name, col in [
            ("Asst. Prof. Jyoti Chauhan", "#1f6feb"),
            ("Asst. Prof. Pradnya Kharade", "#388bfd"),
            ("Dr. Mahendra Kanojia", "#3fb950"),
            ("Asst. Prof. Rohit Sahu", "#d29922"),
            ("Asst. Prof. Mukesh Verma", "#f78166"),
            ("Asst. Prof. Chetana Tanavade", "#8957e5"),
            ("Asst. Prof. Chetan Kanojia", "#f85149"),
        ]:
            tk.Label(fac, text=f"  {name}  ", bg=col, fg='white',
```

```
                font=('Segoe UI', 8), relief='flat').pack(side='left', padx=3)

    #  TAB: GRADES  (Student view)
    def _tab_grades(self, p):
        C = self.C

        tk.Label(p, text="📊  Grade Tracker — FYCS Sem I",
                bg=C['bg'], fg=C['accent'], font=('Segoe UI', 17, 'bold')).pack(anchor='w', padx=20,
pady=14)

        ib = _card(p, C)
        ib.pack(fill='x', padx=20, pady=(0, 10))

        tk.Label(ib, text="Subject:", bg=C['card'], fg=C['text_dim'],
                font=('Segoe UI', 11)).pack(side='left', padx=6)
        self._gsj = tk.StringVar(value=self.SUBJECTS[0])
        ttk.Combobox(ib, textvariable=self._gsj, values=self.SUBJECTS,
                    state='readonly', font=('Segoe UI', 11), width=34).pack(side='left', padx=6)
        tk.Label(ib, text="Marks:", bg=C['card'], fg=C['text_dim'],
                font=('Segoe UI', 11)).pack(side='left', padx=6)
        self._gm = _entry(ib, C, width=7)
        self._gm.pack(side='left', padx=4, ipady=4)
        tk.Label(ib, text="Out of:", bg=C['card'], fg=C['text_dim'],
                font=('Segoe UI', 11)).pack(side='left', padx=6)
        self._gmx = _entry(ib, C, width=7)
        self._gmx.insert(0, '100')
        self._gmx.pack(side='left', padx=4, ipady=4)
        ModernButton(ib, "Save", self._save_grade,`
                    color=C['success'], font=('Segoe UI', 10), padx=12, pady=6).pack(side='left',
padx=10)

        pane = tk.Frame(p, bg=C['bg'])
        pane.pack(fill='both', expand=True, padx=20, pady=4)

        left = tk.Frame(pane, bg=C['bg'])
        left.pack(side='left', fill='both', expand=True, padx=(0, 8))

        self._gt = ttk.Treeview(left,
                            columns=('subject', 'marks', 'max', 'pct', 'grade', 'updated'),
                            show='headings', height=20)
        for col, txt, w in [('subject', 'Subject', 225), ('marks', 'Marks', 70),
                        ('max', 'Out of', 65), ('pct', '%', 65),
                        ('grade', 'Grade', 65), ('updated', 'Updated', 130)]:
            self._gt.heading(col, text=txt)
```

```
        self._gt.column(col, width=w, anchor='center')
    vsb = ttk.Scrollbar(left, orient='vertical', command=self._gt.yview)
    self._gt.configure(yscrollcommand=vsb.set)
    vsb.pack(side='right', fill='y')
    self._gt.pack(fill='both', expand=True)

    sc = _card(pane, C, width=245)
    sc.pack(side='right', fill='y')
    sc.pack_propagate(False)
    self._gsl = tk.Label(sc, text="", bg=C['card'], fg=C['text'],
                         font=('Consolas', 11), justify='left', anchor='nw')
    self._gsl.pack(fill='both', expand=True, padx=10, pady=10)
    self._ref_grades()

@staticmethod
def _letter(pct):
    if pct >= 90: return 'O'
    if pct >= 80: return 'A+'
    if pct >= 70: return 'A'
    if pct >= 60: return 'B+'
    if pct >= 50: return 'B'
    if pct >= 40: return 'C'
    return 'F'

def _save_grade(self):
    subj = self._gsj.get().strip()
    try:
        m  = float(self._gm.get().strip())
        mx = float(self._gmx.get().strip())
    except ValueError:
        messagebox.showerror("Error", "Marks must be numbers."); return
    if not subj: messagebox.showerror("Error", "Select a subject."); return
    if not (0 <= m <= mx):
        messagebox.showerror("Error", "Marks must be between 0 and max."); return
    self.db.upsert_grade(self.current_user['id'], subj, m, mx)
    self._gm.delete(0, tk.END)
    self._ref_grades()

def _ref_grades(self):
    for r in self._gt.get_children(): self._gt.delete(r)
    rows = self.db.get_grades(self.current_user['id'])
    pcts = []
    for r in rows:
        pct = (r['marks'] / r['max_marks']) * 100 if r['max_marks'] else 0
```

```
        lg  = self._letter(pct)
        self._gt.insert('', 'end',
                    values=(r['subject'], f"{r['marks']:.1f}",
                            f"{r['max_marks']:.0f}", f"{pct:.1f}%",
                            lg, r['updated_at']))
        pcts.append(pct)

    if pcts:
        avg = sum(pcts) / len(pcts)
        txt = (
            f"  Statistics\n\n"
            f"  Subjects : {len(pcts)}\n"
            f"  Average  : {avg:.1f}%\n"
            f"  Highest  : {max(pcts):.1f}%\n"
            f"  Lowest   : {min(pcts):.1f}%\n\n"
            f"  Grade Scale:\n"
            f"  O   (>=90%) : {sum(1 for g in pcts if g >= 90)}\n"
            f"  A+  (>=80%) : {sum(1 for g in pcts if 80 <= g < 90)}\n"
            f"  A   (>=70%) : {sum(1 for g in pcts if 70 <= g < 80)}\n"
            f"  B+  (>=60%) : {sum(1 for g in pcts if 60 <= g < 70)}\n"
            f"  B   (>=50%) : {sum(1 for g in pcts if 50 <= g < 60)}\n"
            f"  C   (>=40%) : {sum(1 for g in pcts if 40 <= g < 50)}\n"
            f"  F   (<40%)  : {sum(1 for g in pcts if g < 40)}\n"
        )
    else:
        txt = "  No grades yet.\n\n  Use the form above\n  to add marks."
    self._gsl.config(text=txt)

#  TAB: DIRECTORY  (Student view)
def _tab_dir(self, p):
    C = self.C

    hf = tk.Frame(p, bg=C['bg'])
    hf.pack(fill='x', padx=20, pady=12)
    tk.Label(hf, text="👥  Student Directory — FYCS",
             bg=C['bg'], fg=C['accent'], font=('Segoe UI', 17, 'bold')).pack(side='left')

    bar = _card(p, C)
    bar.pack(fill='x', padx=20, pady=(0, 10))
    tk.Label(bar, text="Search:", bg=C['card'], fg=C['text_dim'],
             font=('Segoe UI', 11)).pack(side='left', padx=8)
    self._ds = tk.StringVar()
    se = _entry(bar, C, textvariable=self._ds, width=34)
    se.pack(side='left', padx=6, ipady=5)
```

```
    se.bind('<KeyRelease>', lambda e: self._ref_dir())

    ModernButton(bar, "Delete", self._del_student,
                 color=C['danger'], font=('Segoe UI', 10), pady=6).pack(side='right', padx=6)
    ModernButton(bar, "Edit", self._edit_student,
                 color=C['warning'], font=('Segoe UI', 10), pady=6).pack(side='right', padx=4)

    self._drt = ttk.Treeview(p,
                             columns=('id', 'roll', 'name', 'phone', 'year', 'dept', 'created'),
                             show='headings', height=22)
    for col, txt, w in [('id', 'Student ID', 90), ('roll', 'Roll No', 75),
                        ('name', 'Full Name', 200), ('phone', 'Phone', 120),
                        ('year', 'Year', 55), ('dept', 'Dept', 55),
                        ('created', 'Registered', 140)]:
        self._drt.heading(col, text=txt)
        self._drt.column(col, width=w)
    vsb = ttk.Scrollbar(p, orient='vertical', command=self._drt.yview)
    self._drt.configure(yscrollcommand=vsb.set)
    vsb.pack(side='right', fill='y', padx=(0, 8))
    self._drt.pack(fill='both', expand=True, padx=20, pady=4)

    self._dc = tk.Label(p, text="", bg=C['bg'], fg=C['text_dim'], font=('Segoe UI', 10))
    self._dc.pack(anchor='e', padx=24, pady=4)
    self._ref_dir()

def _ref_dir(self):
    q = self._ds.get().strip() if hasattr(self, '_ds') else ''
    for r in self._drt.get_children(): self._drt.delete(r)
    rows = self.db.search_students(q) if q else self.db.get_all_students()
    for r in rows:
        self._drt.insert('', 'end',
                         values=(r['id'], r['roll_no'], r['name'], r['phone'],
                                 r['year'], r['department'], r['created_at']))
    self._dc.config(text=f"{len(rows)} student(s) found")

def _del_student(self):
    sel = self._drt.selection()
    if not sel: messagebox.showinfo("Info", "Select a student to delete."); return
    sid = self._drt.item(sel[0])['values'][0]
    if sid == self.current_user['id']:
        messagebox.showerror("Error", "You cannot delete your own account."); return
    if messagebox.askyesno("Confirm", f"Delete student {sid}? This cannot be undone."):
        self.db.delete_student(sid)
        self._ref_dir()
```

```
    def _edit_student(self):
        sel = self._drt.selection()
        if not sel: messagebox.showinfo("Info", "Select a student to edit."); return
        sid = self._drt.item(sel[0])['values'][0]
        row = self.db.get_student(sid)
        if not row: return
        self._open_edit_student_window(sid, row, self._ref_dir)

    #  THEME / LOGOUT / CLOSE
    def _toggle_theme(self):
        self.dark_mode = not self.dark_mode
        self.C = DARK if self.dark_mode else LIGHT
        self._style()
        if self.current_user:
            if self.is_admin: self._build_admin_dashboard()
            else: self._build_dashboard()
        else:
            self._build_login()

    def _logout(self):
        if messagebox.askyesno("Logout", "Are you sure you want to logout?"):
            self.current_user = None
            self.is_admin = False
            self._build_login()

    def on_close(self):
        self.db.close()
        self.root.destroy()


#  ENTRY POINT
def main():
    root = tk.Tk()
    app  = CampusConnect(root)
    root.protocol("WM_DELETE_WINDOW", app.on_close)
    root.mainloop()

if __name__ == "__main__":
    main()
```